Database Design and Normalization: The Sick Database

Isin-325

Ethan Krul

1/27/2026

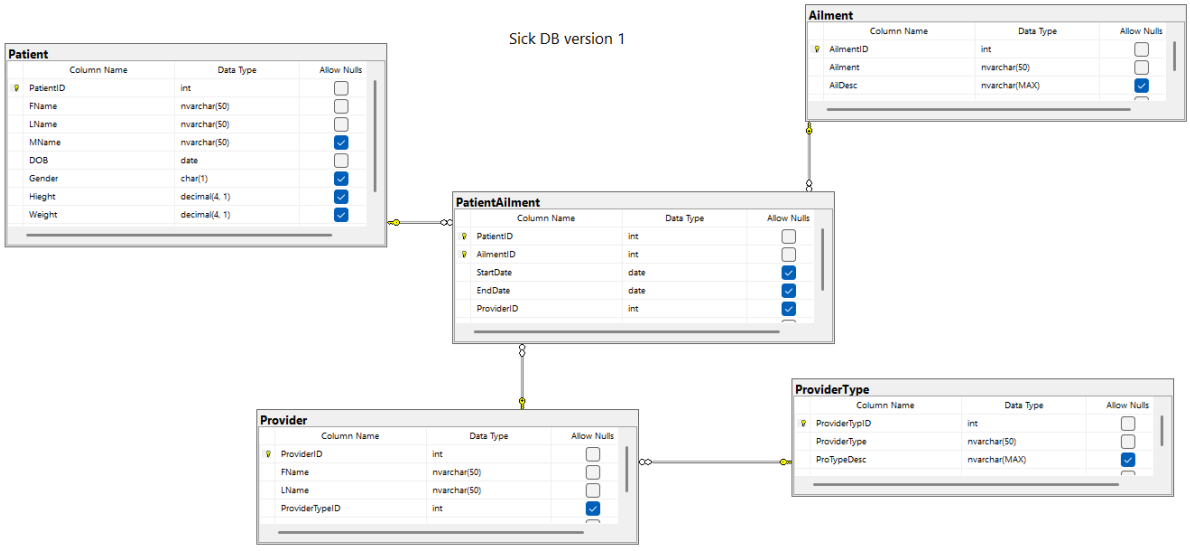
Relational Database Theory (Normalization)

Relational database theory provides the foundation for designing efficient and reliable databases. It organizes data into tables that are related through primary and foreign keys. Normalization is the process of structuring tables to reduce redundancy and prevent data anomalies. First Normal Form requires that all attributes be atomic and contain no repeating groups. Second Normal Form ensures that all non-key attributes depend on the entire primary key. Third Normal Form requires that non-key attributes depend only on the primary key and not on other non-key attributes.

Database Diagram and Tables

The Sick database is designed to store and manage information about patients, their medical ailments, and the healthcare providers who treat them. The system tracks patient demographic information, documented ailments, and the relationship between patients and providers over time. Each entity (Patient, Provider, Ailment) is stored in its own table, and relationships between entities are handled using foreign keys.

Sick Database Entity Relationship Diagram



Tables

Patient:

Stores demographic and physical information about each patient.

Primary Key: PatientId

Includes first name, last name, date of birth, gender, height, and weight.

All fields are atomic and appropriately typed.

Ailment: Stores medical conditions that may be diagnosed for patients.

Primary Key: AilmentId

Includes ailment name and description.

Provider:

Stores information about healthcare providers.

Primary Key: ProviderId

Includes provider name and provider type.

Uses a foreign key to ProviderType.

ProviderType:

Defines categories of healthcare providers.

Primary Key: ProviderTypeId

Examples include physician, nurse, or specialist.

PatientAilment:

Acts as a junction table connecting patients, ailments, and providers.

Composite primary key Relationship: PatientId and AilmentId

Includes start and end dates of treatment.

Enforces many-to-many relationships without redundancy.

Keys, Data Types, and Referential Integrity

Each table in the Sick database includes defined primary keys and the appropriate data types. Integer data types are used for primary and foreign keys to support indexing and relationships.

Text-based attributes use nvarchar to support character storage, while date and decimal data types are used for birth dates, treatment dates, height, and weight.

Referential integrity is enforced through the use of foreign key constraints. For example, PatientAilment.PatientId references Patient.PatientId, PatientAilment.AilmentId references Ailment.AilmentId, and Provider.ProviderTypeId references ProviderType.ProviderTypeId. This ensures that related records cannot exist without valid parent records, preventing lost data and maintaining consistency.

Database Standards
All tables and columns follow the CamelCase naming convention. Columns are atomic and contain single values only. Appropriate SQL data types are used to for performance and data integrity. Referential integrity is enforced using foreign key constraints rather than application logic or queries. The database is designed for OLTP usage, and index usage is limited to primary and foreign keys.

Violations of the Three Normal Forms

First Normal Form Violation - A table stores multiple values in a single column: Ailments = 'Diabetes, Asthma, Hypertension'

This is a problem because it makes searching, updating, and validating data difficult and can lead to inconsistent formatting. It also leads to repeating groups because multiple ailments are stored in a single field. To correct it, you can separate ailments into their own table and use a junction table to link patients to ailments.

The Second Normal Form Violation - A junction table includes name: PatientId | AilmentId | PatientName

This breaks the rule since PatientName depends on PatientId, not on the full composite key and creates a partial dependency. To fix this you move PatientName to Patient table, where the it fully depends on the primary key.

The Third Normal Form Violation - A Provider table storing ProviderTypeName: ProviderId | ProviderTypeId | ProviderTypeName

This breaks the rule because ProviderTypeName depends on the ProviderTypeId, not directly on ProviderID. This creates a transitive dependency, to fix this you move the ProviderTypeName to a separate ProviderType table and reference it using a foreign key.